

Problem 3 — Many-Time Pad Encryption

Encrypt ten predefined ASCII messages using one shared key. Save plaintexts, key, and ciphertexts as hexadecimal strings in one JSON file.

The messages below are editable examples. Replace them with your own chosen messages if desired. This simulation uses known plaintext and a known key for learning; Problem 4 uses the separately supplied dataset.

Run from `Many_Time_Pad/` or the repository root with Python 3.9 or newer.

```
In [17]: from pathlib import Path
import json
import secrets

MESSAGES = [
    "How many of us are in the know?",
    "I speak namely of John Snow",
    "that would be King in the North.",
    "The Dwarf, the Spider, and",
    "a grand maester too, and an Egg.",
    "It is no secret now, but information",
    "If Her Grace were to learn of it...",
    "Keep silence for now. If gods be good",
    "we shall see the morning alive",
    "and not be fodder for dragons.",
]

workspace = Path("Many_Time_Pad") if Path("Many_Time_Pad").is_dir() else Path(".")
(workspace / "data").mkdir(parents=True, exist_ok=True)

if len(MESSAGES) != 10 or any(not message for message in MESSAGES):
    raise ValueError("Provide exactly ten nonempty messages.")

plaintexts = [message.encode("ascii") for message in MESSAGES]
```

One key reused across all ten messages

To ensure message integrity throughout the encoding stream, we must generate enough bytes for the longest message. Each message starts with key byte zero and uses the matching prefix. Re-running with a new key changes the ciphertexts, but the XOR of two ciphertexts remains the XOR of their plaintexts.

```
In [8]: def encrypt_with_shared_key(plaintexts, key):
    if not plaintexts or len(key) < max(map(len, plaintexts)):
        raise ValueError("The shared key must cover the longest message.")
    return [bytes(byte ^ key[offset] for offset, byte in enumerate(plaintext))
            for plaintext in plaintexts]
```

```
# Generate once, then reuse this same key for all ten messages.
key = secrets.token_bytes(max(map(len, plaintexts)))
ciphertexts = encrypt_with_shared_key(plaintexts, key)

for index, (plaintext, ciphertext) in enumerate(zip(plaintexts, ciphertexts), 1):
    recovered = bytes(byte ^ key[offset] for offset, byte in enumerate(ciphertext))
    assert recovered == plaintext, f"Round-trip failure in message {index}."
    print(f"M{index:02d}: {plaintext.decode('ascii')}")
    print("Plaintext hex:", plaintext.hex())
    print("Ciphertext hex:", ciphertext.hex())
print("Shared key hex:", key.hex())
```

M01: How many of us are in the know?
 Plaintext hex: 486f77206d616e79206f6662075732061726520696e20746865206b6e6f773f
 Ciphertext hex: 1a42e5b9ad23adb34e240abe2ad5f26a7fce747017fd44a19886f6645e7363

M02: I speak namely of John Snow
 Plaintext hex: 4920737065616b206e616d656c79206f666204a6f686e20536e6f77
 Ciphertext hex: 1b0de1e9a523a8ea002a01fb33dff2646b8b1e7611b3109a93c9ea

M03: that would be King in the North.
 Plaintext hex: 7468617420776f756c64206265204b696e6720696e20746865204e6f7274682e
 Ciphertext hex: 2645f3ede035acbf022f4cfc3a86996263cc747017fd44a19886d3654370349a

M04: The Dwarf, the Spider, and
 Plaintext hex: 5468652044776172662c20746865205370696465722c20616e64
 Ciphertext hex: 0645f7b98435a2b808674cea37c3f2587dc2307c0bf110a893c2

M05: a grand maester too, and an Egg.
 Plaintext hex: 61206772616e64206d61657374657220746f6f2c20616e6420616e204567672e
 Ciphertext hex: 330df5eba12ca7ea032a09ed2bc3a02b79c43b3559bc5eaddc7f32a74633b9a

M06: It is no secret now, but information
 Plaintext hex: 4974206973206e6f20736563726574206e6f772c2062757420696e666f726d6174696
 f6e
 Ciphertext hex: 1b59b2f0b362ada54e3809fd2dc3a62b63c4233559bf45bdddcff36c5e7631d5491a
 016a

M07: If Her Grace were to learn of it...
 Plaintext hex: 496620486572204772616365207765726520746f206c6561726e206f662069742e2e2
 e
 Ciphertext hex: 1b4bb2d1a530e38d1c2a0ffb7fd1b779688b207659b155a88fc8bd65572435c0135d
 40

M08: Keep silence for now. If gods be good
 Plaintext hex: 4b6565702073696c656e636520666f72206e6f772e20496620676f647320626520676
 f6f64
 Ciphertext hex: 1948f7e9e031aaa60b250ffb7fc0bd792dc53b6e57fd79afddc1f26e42243ed11d14
 016b07

M09: we shall see the morning alive
 Plaintext hex: 7765207368616c6c2073656520746865206d6f726e696e6720616c697665
 Ciphertext hex: 2548b2eaa823afa64e3809fb7fd2ba6e2dc63b6b17b45eaeddc7f1634761

M10: and not be fodder for dragons.
 Plaintext hex: 616e64206e6f7420626520666f6464657220666f7220647261676f6e732e
 Ciphertext hex: 3343f6b9ae2db7ea0c2e4cf830c2b66e7f8b32760bfd54bb9cc1f264422a
 Shared key hex: 522d9299c042c3ca6e4b6c9e5fa6d20b0dab541979dd30c9fda69d0a31045cb43d73
 6e0463

Save the required hexadecimal artifact

This cell writes only `data/simulation.json`; it does not modify the official Problem 4 inputs. A rerun replaces the simulation artifact. All plaintext, key, and ciphertext values are stored in hexadecimal form; labels and lengths are metadata.

Copies of this notebook and related notebooks for this project will be kept on my GitHub at: https://github.com/joe-cjr10071-nyu/AppCrypto/blob/main/Many_Time_Pad/P3_Many_Time_Pad_Encrypt.ipynb

```
In [9]: artifact = {
    "exercise": "Problem 3 simulation",
    "encoding": "ASCII plaintext; hex byte values",
    "key_hex": key.hex(),
    "messages": [
        {
            "label": f"M{index:02d}",
            "byte_length": len(plaintext),
            "plaintext_hex": plaintext.hex(),
            "ciphertext_hex": ciphertext.hex(),
        }
        for index, (plaintext, ciphertext) in enumerate(zip(plaintexts, ciphertexts))
    ],
}
OUTPUT_FILE = workspace / "data" / "simulation.json"
OUTPUT_FILE.write_text(json.dumps(artifact, indent=2) + "\n", encoding="ascii")
print(f"Saved ten messages, one shared key, and ten ciphertexts to {OUTPUT_FILE}")
```

Saved ten messages, one shared key, and ten ciphertexts to data/simulation.json

Observe key cancellation

Here we can see the impact of changing the compared messages and their respective keys/ciphers when they are XOR'ed.

Change LEFT_MESSAGE and RIGHT_MESSAGE to compare a different pair. Test modified plaintexts or a different key and record your observations for the written report.

One thing remains constant: *The ciphertext XOR and the plaintext XOR **are equal** when the same shared key are used.*

```
In [16]: LEFT_MESSAGE = 2
RIGHT_MESSAGE = 8
if not (1 <= LEFT_MESSAGE <= 10 and 1 <= RIGHT_MESSAGE <= 10):
    raise ValueError("Choose message numbers from 1 through 10.")
if LEFT_MESSAGE == RIGHT_MESSAGE:
    raise ValueError("Select two different messages.")

def xor_overlap(left, right):
    return bytes(a ^ b for a, b in zip(left, right))

left = LEFT_MESSAGE - 1
right = RIGHT_MESSAGE - 1
```

```

ciphertext_xor = xor_overlap(ciphertexts[left], ciphertexts[right])
plaintext_xor = xor_overlap(plaintexts[left], plaintexts[right])
assert ciphertext_xor == plaintext_xor

print("Ciphertext XOR:", ciphertext_xor.hex())
print("Plaintext XOR: ", plaintext_xor.hex())
print("They match: the reused key cancels over the overlapping positions.")

alternative_key = secrets.token_bytes(len(key))
alternative_ciphertexts = encrypt_with_shared_key(plaintexts, alternative_key)
assert xor_overlap(alternative_ciphertexts[left], alternative_ciphertexts[right]) =
print("The same relation also holds after changing the shared key.")

```

Ciphertext XOR: 024516004512024c0b0f0e004c1f4f1d464e2518464e69354e0818

Plaintext XOR: 024516004512024c0b0f0e004c1f4f1d464e2518464e69354e0818

They match: the reused key cancels over the overlapping positions.

The same relation also holds after changing the shared key.

Observations

- Changing the pads changes the ciphertext; the plain text remains invariant (assuming no different messages are generated).
- But if the messages are otherwise unchanged, leaked associations for what has been partially decoded or strongly suggestive of a language pattern can still be used to infer further letter swaps and bit flips.
- XORing ciphertexts sharing the same letter in the same spot zero out, indicating they share a byte at that location.
- a letter XORed a space character will flip its case (in ASCII).